

CodeGuild

Fraternidade Acadêmica de Desenvolvedores

Documento de Proposta e Escopo - Entrega 1

Disciplina	Desenvolvimento de Software para Web
Founders	Matheus Vasco, Pedro Simon, Erick Alves, Guilherme Leal e Guilherme Silva
Conceito central	Conectar estudantes por competências técnicas para formar equipes, desenvolver projetos e criar experiência prática.

Transformando interesses técnicos em equipes, projetos reais e aprendizado colaborativo.

1. Visão do projeto

A CodeGuild é uma comunidade acadêmica voltada a estudantes e desenvolvedores em formação que desejam transformar aprendizado teórico em experiência prática. A plataforma aproxima pessoas por tecnologias, interesses e objetivos, permitindo formar equipes para projetos, encontrar vagas compatíveis e construir portfólio colaborativo.

Problema central

Ideias e chamadas de projetos ficam dispersas em conversas, enquanto alunos têm dificuldade para identificar colegas com stacks e interesses compatíveis. O resultado são equipes que não se formam e projetos que não saem do papel.

1.1 Público-alvo

- Estudantes iniciantes que buscam o primeiro projeto prático e parceiros no mesmo nível.
- Alunos intermediários e veteranos que desejam liderar equipes, praticar novas tecnologias e fortalecer o portfólio.
- Monitores e professores que desejam propor desafios, divulgar iniciativas ou orientar projetos acadêmicos.

1.2 Proposta de valor

A CodeGuild resolve o "momento zero" da colaboração: descobrir pessoas e oportunidades adequadas antes de migrar o trabalho para ferramentas como GitHub, Discord ou Trello. No lançamento, a comunidade oferecerá perfil técnico, mural de projetos e vagas, candidaturas, match simples de compatibilidade e integração com dados públicos do GitHub.

Hipótese de validação

O campus funciona como um ambiente acessível para recrutar usuários reais, observar comportamento, aplicar testes de usabilidade e ajustar o produto com base em feedback contínuo.

2. Escopo do MVP

O escopo foi dividido por entrega para evitar excesso de funcionalidades e garantir que o núcleo do produto esteja funcional até o final do semestre.

2.1 Entrega 2 - Landing Page

Área	Será entregue	Objetivo
Identidade	Logo/tipografia, paleta, slogan e linguagem visual	Apresentar a marca CodeGuild de forma consistente.
Hero	Resumo do problema, proposta de valor e CTA	Explicar em poucos segundos o motivo para participar.
Problema e solução	Blocos curtos com as dores e o funcionamento da plataforma	Convencer o visitante de que a comunidade resolve uma necessidade real.
Como funciona	Fluxo: criar perfil -> encontrar projeto -> candidatar-se -> formar equipe	Tornar o produto fácil de entender.
Público e benefícios	Benefícios para iniciantes, veteranos e professores	Mostrar valor para diferentes perfis.
Pré-cadastro	Formulário com nome, e-mail, curso/período e áreas de interesse	Capturar interessados e validar demanda.
Responsividade	Compatibilidade com desktop e celular	Atender ao requisito de experiência web

Área	Será entregue	Objetivo
		moderna.

Fora da Entrega 2: autenticação, criação real de projetos, candidaturas, painel de usuário e integração com GitHub. Esses itens pertencem à Entrega 3.

2.2 Entrega 3 - Plataforma funcional

Prioridade	Módulo	Funcionalidades	Critério de conclusão
MUST	Autenticação	Cadastro, login, logout e proteção de rotas	Usuário consegue criar conta e acessar apenas sua área autenticada.
MUST	Perfil técnico	Bio, nível, tecnologias dominadas/estudadas, interesses e GitHub username	Perfil pode ser criado, editado e consultado.
MUST	GitHub	Consulta de perfil público, repositórios e linguagens	Dados externos são exibidos com tratamento de erro e atualização controlada.
MUST	Projetos	Criar, editar, listar e encerrar projeto	Responsável gerencia seu projeto e visitantes autenticados conseguem descobri-lo.
MUST	Vagas	Criar vagas por especialidade/stack e quantidade	Projeto pode anunciar perfis necessários para formar a equipe.
MUST	Candidaturas	Candidatar-se, aprovar, rejeitar e cancelar candidatura	Fluxo completo de formação de equipe funciona.
SHOULD	Match	Percentual simples com base em tecnologias coincidentes	Plataforma exibe aderência aproximada sem IA.
SHOULD	Grupos de estudo	Criar grupo, entrar/sair e listar membros	Comunidade oferece colaboração além dos projetos sem criar um fórum complexo.
COULD	Filtros	Buscar projetos por tecnologia, nível e status	Melhora descoberta se houver tempo após o núcleo estar estável.
COULD	Admin básico	Gerenciar catálogo de tecnologias e moderar projetos inadequados	Mantém dados essenciais organizados.

2.3 Explicitamente fora do MVP

- Chat ou mensagens em tempo real.
- Execução de código dentro da plataforma.
- Inteligência artificial para recomendação ou avaliação de candidatos.
- Aplicativo mobile nativo.
- Pagamentos, assinaturas ou marketplace.
- Videoconferência, mentoria completa ou sistema de cursos.
- Notificações em tempo real por push/e-mail.
- Substituição de GitHub, Discord, Jira ou Trello: a CodeGuild apenas facilita a formação inicial das equipes.

Decisão de escopo

O sucesso do MVP será medido pela capacidade de um estudante criar um perfil, descobrir um projeto compatível, candidatar-se a uma vaga e ser aceito na equipe. Funcionalidades que não ajudam esse fluxo ficam em segundo plano.

3. Arquitetura e stack proposta

A arquitetura recomendada é uma aplicação web cliente-servidor com API REST e banco relacional. O objetivo é privilegiar simplicidade, separação de responsabilidades e facilidade de manutenção, sem introduzir microserviços ou infraestrutura desnecessária.

Camada	Tecnologias	Responsabilidade
Front-end	HTML5, CSS3, Bootstrap 5 e JavaScript	Landing page, formulários, navegação, consumo da API e interface responsiva.
Back-end	Python + FastAPI	Regras de negócio, autenticação, endpoints REST, validações e integração externa.
Persistência	PostgreSQL + SQLAlchemy	Usuários, perfis, tecnologias, projetos, vagas, candidaturas e grupos.
Autenticação	JWT + hash seguro de senha	Sessões stateless e proteção de rotas privadas.
API externa	GitHub REST API	Importar dados públicos do desenvolvedor e enriquecer o perfil técnico.
Versionamento	Git + GitHub	Controle de versão, branches, pull requests e histórico do projeto.
Gestão	Jira ou Trello	Backlog, acompanhamento do fluxo e organização das entregas.

3.1 Visão arquitetural



A API FastAPI também consulta a GitHub REST API. Para reduzir dependência externa, os dados essenciais podem ser armazenados com data de última atualização e atualizados sob demanda.

4. Regras de negócio essenciais

- Um usuário pode manter apenas uma candidatura ativa para a mesma vaga.
- Apenas o responsável pelo projeto pode criar/editar vagas e aprovar ou rejeitar candidatos.
- Uma vaga encerrada ou sem disponibilidade não aceita novas candidaturas.
- O match é apenas informativo e usa tecnologias coincidentes; ele não decide automaticamente quem entra no projeto.
- O GitHub username é opcional. A ausência ou indisponibilidade da API não impede o uso das demais funções.
- Um usuário pode participar de vários projetos, respeitando as regras de cada vaga.
- Dados sensíveis como senha nunca serão armazenados em texto puro.

5. Requisitos não funcionais mínimos

Requisito	Aplicação prática
Responsividade	Telas utilizáveis em desktop e dispositivos móveis.
Segurança básica	Hash de senha, validação de entrada, autorização e proteção das rotas privadas.
Desempenho	Listagens paginadas e chamadas externas controladas.
Resiliência	Erro da GitHub API gera mensagem amigável e não derruba a plataforma.
Usabilidade	Fluxos curtos, feedback visual de ações e formulários com mensagens de validação.

Requisito	Aplicação prática
Manutenibilidade	Código organizado por responsabilidades, padrões de nome e documentação mínima do projeto.

6. Critérios de sucesso do MVP

- Usuário consegue concluir cadastro e montar perfil técnico sem ajuda da equipe.
- Usuário consegue localizar pelo menos um projeto e entender claramente suas vagas.
- Candidatura passa pelos estados PENDENTE -> APROVADA ou REJEITADA sem inconsistências.
- Responsável do projeto visualiza candidatos e forma sua equipe pela plataforma.
- Integração com GitHub adiciona valor ao perfil, mas o sistema continua funcional se a API estiver indisponível.
- Landing page comunica problema, solução e chamada para ação de forma clara em menos de um minuto.

7. Roadmap resumido

Fase	Foco	Saída esperada	Validação
Entrega 1	Ideia, pitch, escopo e gestão	Proposta formal + quadro configurado	Clareza e viabilidade.
Entrega 2	Aquisição e interesse	Landing responsiva + pré-cadastro	Feedback e intenção de uso.
Entrega 3 - núcleo	Perfil + projetos + vagas + candidaturas	MVP funcional	Formação real/simulada de equipe.
Entrega 3 - refinamento	GitHub, match, grupos e filtros	Experiência enriquecida	Testes de usabilidade e correções.

8. Riscos e controle de escopo

Risco	Impacto	Mitigação
Tentar reproduzir GitHub/Discord/Jira	Escopo cresce e a entrega atrasa.	Manter o foco em descoberta, candidatura e formação de equipes.
GitHub API indisponível ou limitada	Perfis externos deixam de atualizar.	Cache/última atualização e funcionamento independente da API.
Poucos dados para demonstração	Plataforma parece vazia.	Criar seed com usuários, tecnologias, projetos e vagas fictícias.
Integração tardia do grupo	Conflitos e retrabalho perto da entrega.	Pull requests pequenas, revisão contínua e Definition of Done comum.
Funcionalidades secundárias atrasarem o núcleo	MVP incompleto.	Implementar MUST antes de SHOULD/COULD.

Conclusão

A CodeGuild é viável como projeto acadêmico porque possui um problema fácil de observar, usuários acessíveis para validação e um fluxo de negócio suficientemente rico para demonstrar front-end, back-end, banco de dados, autenticação e integração com API externa. O recorte de MVP evita a construção de uma rede social genérica e concentra a entrega na formação de equipes para projetos práticos.

CodeGuild: construa código, conecte pessoas, forme seu futuro.